

큐 (Queue)

강 지 훈

jhkang@cnu.ac.kr



큐 (Queue)

큐 (Queue)

■ 특수한 성격의 리스트:

"서비스를 받기 위해 서는 줄"

- 줄에 먼저 서는 것이 먼저 서비스 받는다.
- 버스를 타기 위해서 서는 줄
- 식당에서 배식을 위해 서는 줄
- 컴퓨터 안에서 일(job)들이 처리 서비스를 받기 위해 서는 줄

■ 큐의 또 다른 용어:

- 선입선출 (先入先出) 리스트
- FIFO (First-In-First-Out) List

큐

■ 순서 리스트 (ordered list):

- 큐 안에 있는 원소의 순서가 삽입과 삭제의 위치를 결정

■ 즉, 삽입과 삭제는 큐의 양 끝에서:

- 새로운 원소의 **삽입**은 항상 큐의 **뒤쪽**에서
- 서비스 받기 위한 원소의 **삭제**는 큐의 **앞쪽**에서

$$Q = (a_0, \dots, a_{n-1})$$



front 원소



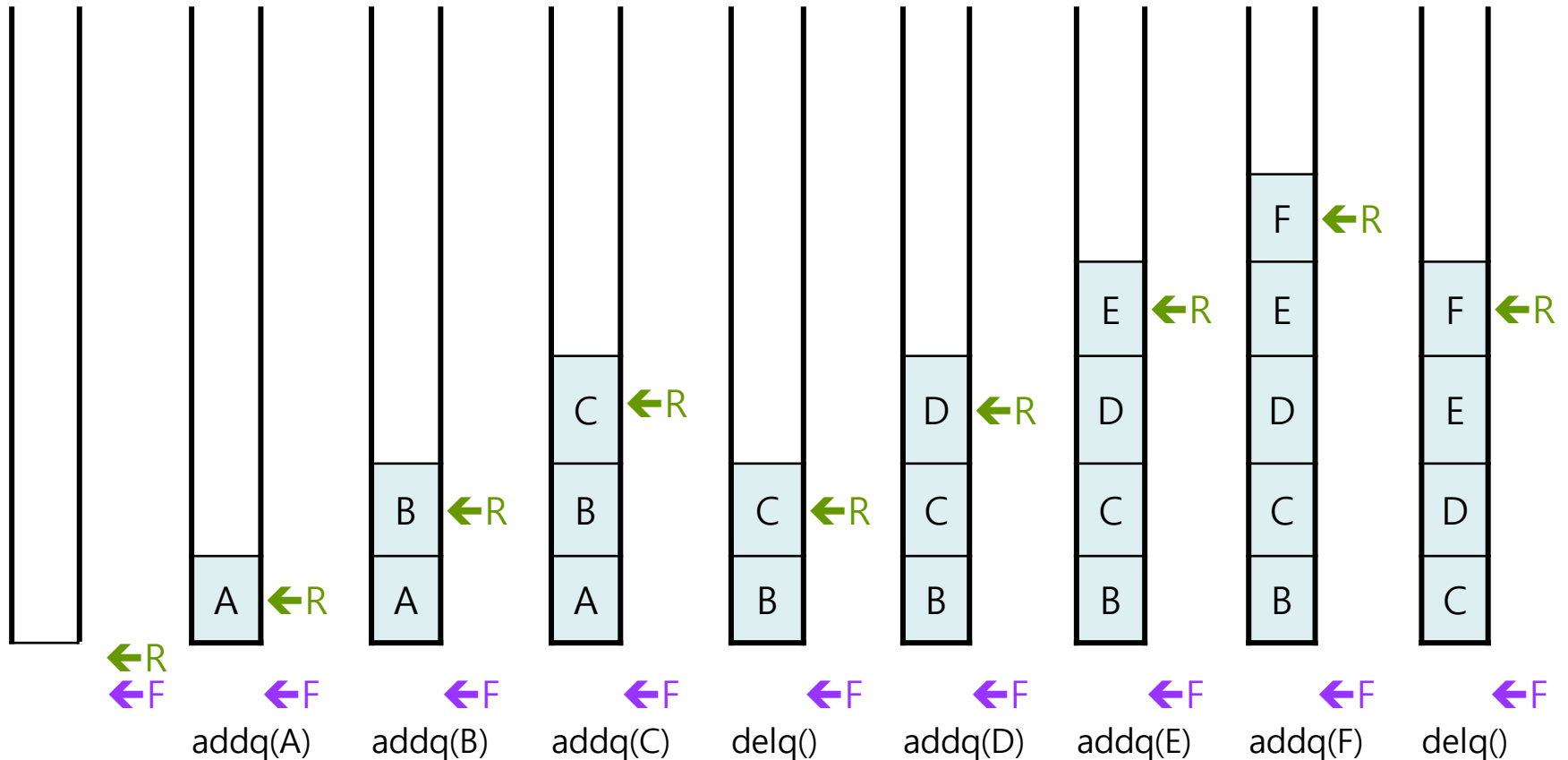
rear 원소

a_{i+1} 은 a_i 의 **뒤**에 있다 ($0 \leq i \leq n-2$)

예: 큐의 작동 원리

"←R" 큐에서 rear 원소를 가리킨다

"←F" 큐에서 front 원소를 가리킨다



Class "ArrayQueue<T>"

❏ ArrayQueue<T>: 공개 함수

■ ArrayQueue<T> 객체 사용법

- public ArrayQueue () ;
- public ArrayQueue (int givenCapacity) ;

- public boolean isEmpty () ;
- public boolean isFull () ;
- public int size () ;

- public T frontElement () ;

- public boolean enqueue (T anElement) ;
- public T dequeue () ;
- public void clear () ;

Class "ArrayQueue"의 구현



□ ArrayQueue: 비공개 인스턴스 변수

```
public class ArrayQueue<T>
{
    // 비공개 인스턴스 변수
    private static final int DEFAULT_CAPACITY = 50 ;
    private int          _capacity ;
    private int          _front ;
    private int          _rear ;
    private T[]          _elements ;
}
```

□ ArrayQueue의 생성자

```
public class ArrayQueue<T>
{
```

```
// 생성자
```

```
public ArrayQueue ()
```

```
{
```

```
    this ArrayQueue.DEFAULT_CAPACITY ;
```

```
}
```

- static으로 선언된 상수/변수/함수를 언급할 때에는 class의 이름으로.
- (static으로 선언되지 않은 것을 언급할 때에는 this)

```
public ArrayQueue (int givenCapacity)
```

```
{
```

```
    @SuppressWarnings("Unchecked") ;
```

```
    this._elements = (T[ ]) new Object[givenCapacity] ;
```

```
    this._capacity = givenCapacity ;
```

```
    this._front = -1 ;
```

```
    this._rear = -1 ;
```

```
}
```

□ ArrayQueue : 상태 알아보기

```
public class ArrayQueue<T>
{
```

.....

```
// 상태 알아보기
```

```
public boolean isEmpty ()
```

```
{
    return (this._front == this._rear) ;
}
```

```
public boolean isFull ()
```

```
{
    return (this._rear == (this._capacity-1) ) ;
}
```

```
public int size()
```

```
{
    return ( this._rear - this._front) ;
}
```

- 큐의 크기 (큐에 들어있는 원소의 개수)를 얻는 getter
- Setter는 존재하지 않는다. 이유는?

❏ ArrayQueue : frontElement()

```
public T frontElement()
{
    T frontElement = null ;
    if ( ! isEmpty() ) {
        frontElement = this._elements[this._front+1] ;
    }
    return frontElement ;
}
```

❑ ArrayQueue : enqueue()

// 원소 추가 함수

```
public boolean enqueue (T anElement)
{
    if ( this.isFull() ) {
        return false ;
    }
    else {
        this._rear++ ;
        this._elements[this._rear] = anElement ;
        return true ;
    }
}
```

❏ ArrayQueue : dequeue()

```
public T dequeue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        this._front++ ;
        frontElement = this._elements[this._front] ;
        this._elements[this._front] = null ;
    }
    return frontElement ;
}
```

❏ ArrayQueue : clear()

```
public void clear ()  
{  
    for ( i = 0 ; i < this.size() ; i++ ) {  
        this._elements[this._front] = null ;  
    }  
    this._front = -1 ;  
    this._rear = -1 ;  
}
```

CircularArrayQueue 로의 구현



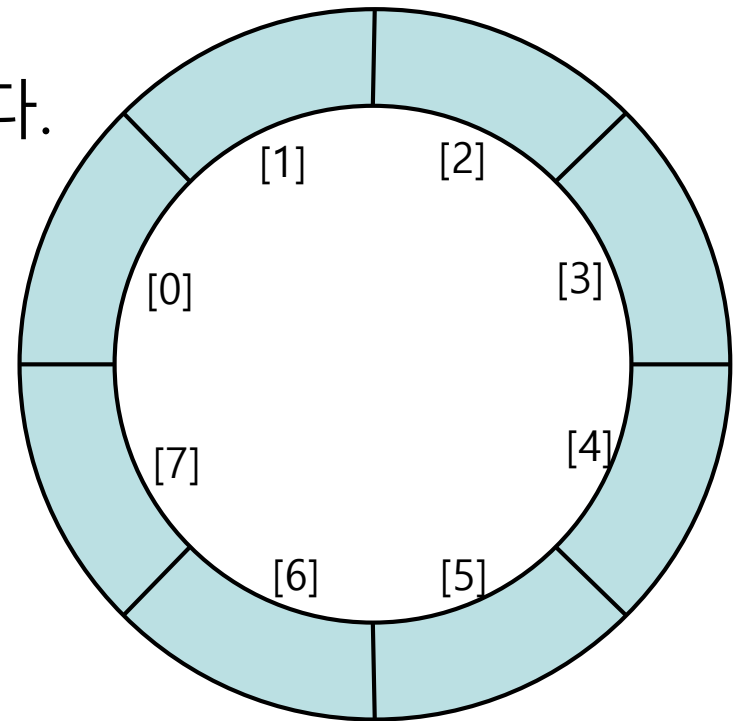
❑ ArrayQueue 구현의 문제점

- 원소의 위치가 점진적으로 왼쪽에서 오른쪽으로 이동!!

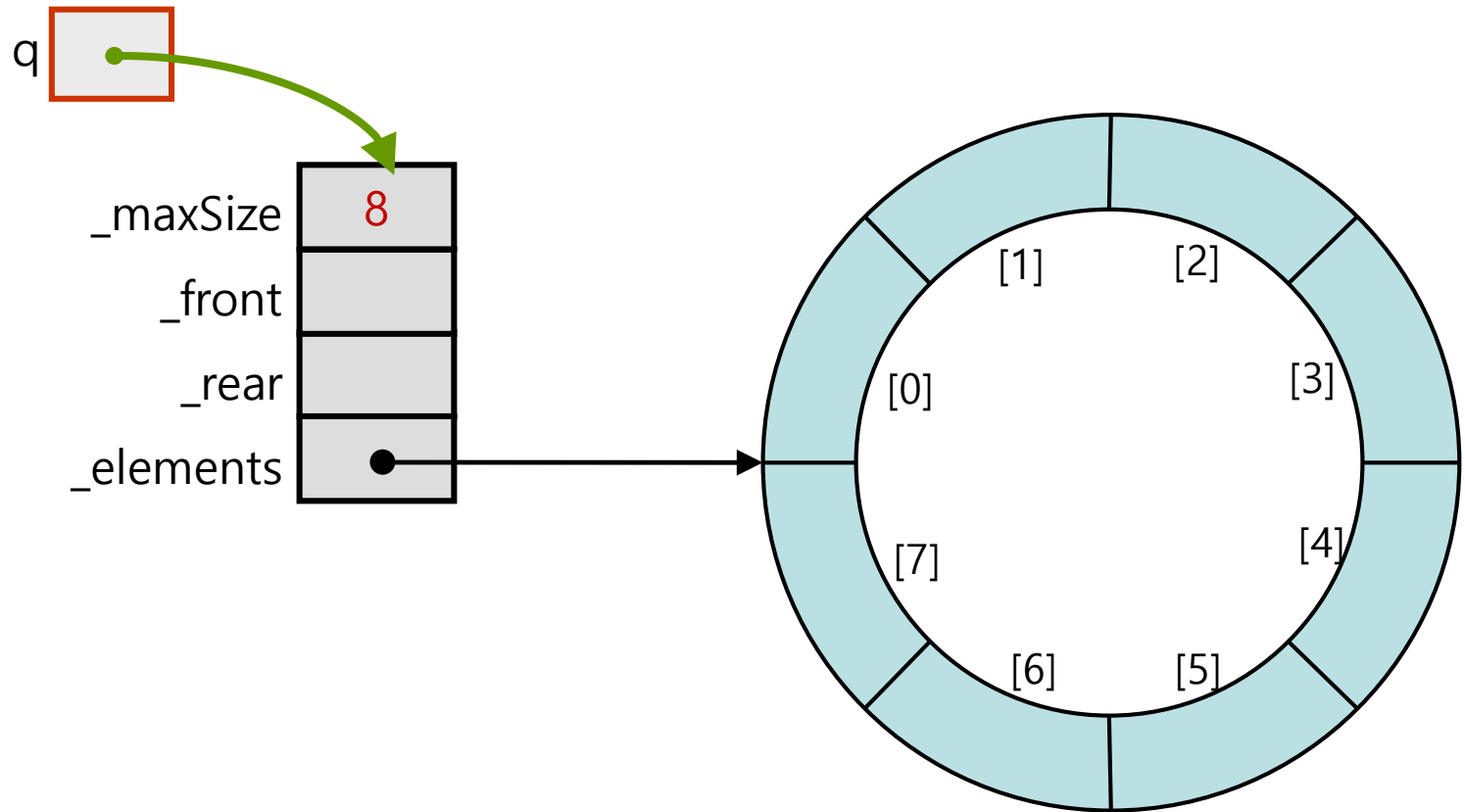
front	rear	Q[0]	Q[1]	Q[2]	Q[3]	설명
-1	-1					Empty queue
-1	0	J_1				enqueue J_1
-1	1	J_1	J_2			enqueue J_2
-1	2	J_1	J_2	J_3		enqueue J_3
0	2		J_2	J_3		dequeue J_1
1	2			J_3		dequeue J_2
1	3			J_3	J_4	enqueue J_4
2	3				J_4	dequeue J_3

□ Circular Array Queue로 구현

- 반지 형태의 환형 배열:
 - 배열의 끝이 배열의 맨 처음과 붙어 있는 것으로 본다.
- Public 함수의 사용법은 바뀌지 않는다!!!
- 함수의 구현만 약간 변경될 뿐이다.
 - 함수 코드를 일부 수정



□ 배열을 반지 형태로 생각하자



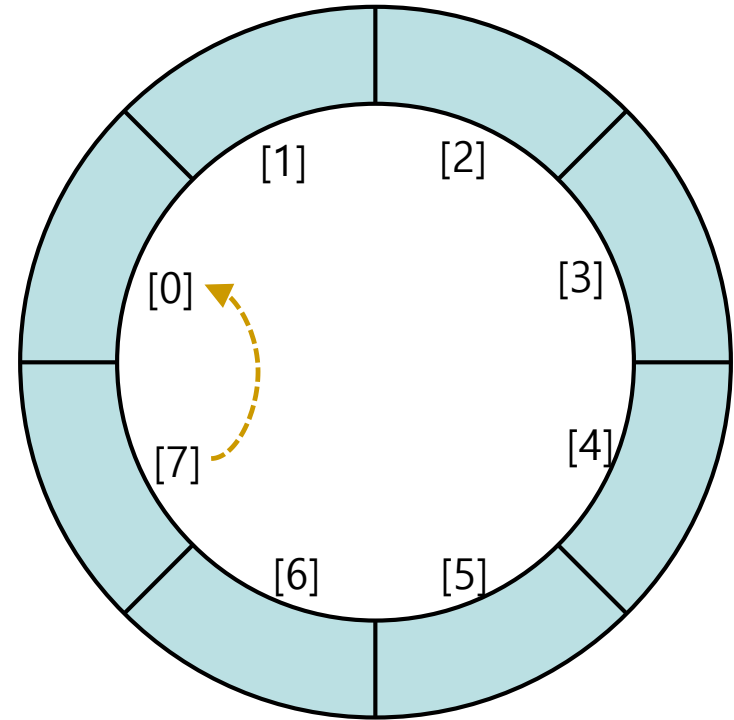
□ 다음 위치 계산은?

- 크기가 8인 Circular Queue 에서,
 - [7]번째 다음은 [0]번째

- 일반적으로 다음 위치 계산은 어떻게?

```
this._rear++;  
if (this._rear == this._capacity)  
    this._rear = 0 ;
```

- 이렇게도...
$$\text{this._rear} = (\text{this._rear} + 1) \% \text{this._capacity} ;$$



□ 초기화 / Empty 조건

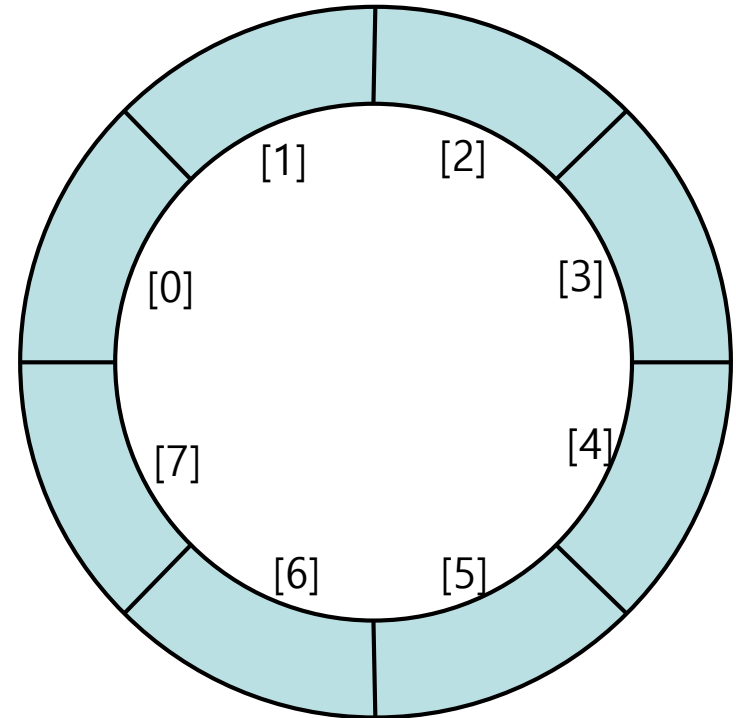
■ 초기화는?

- `this._front = 0 ;`
- `this._rear = 0 ;`

■ 큐 Empty 조건은?

- `(this._front == this._rear)`
 - ◆ 초기 상태도 empty임

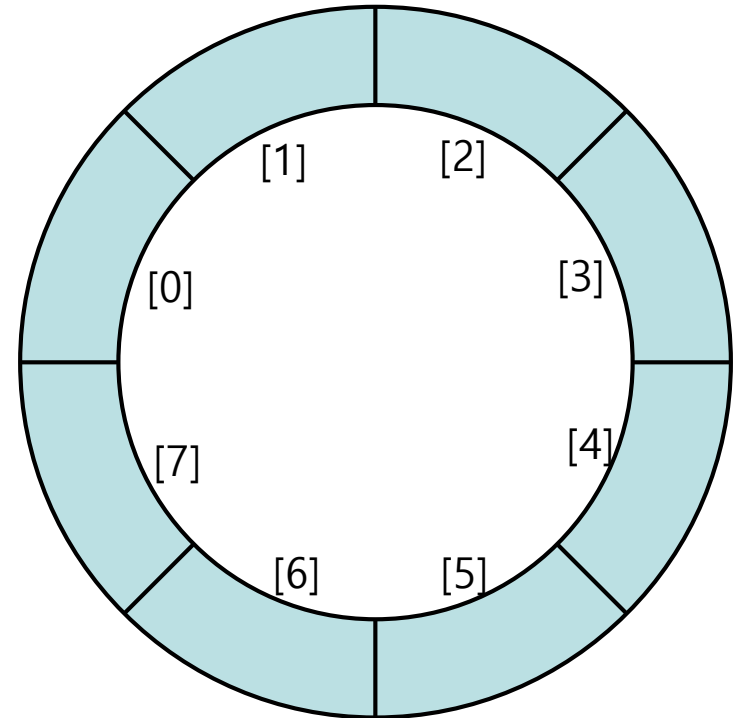
■ 큐 Full 조건은?



□ Full 조건 [1]

■ 큐 Full 조건은?

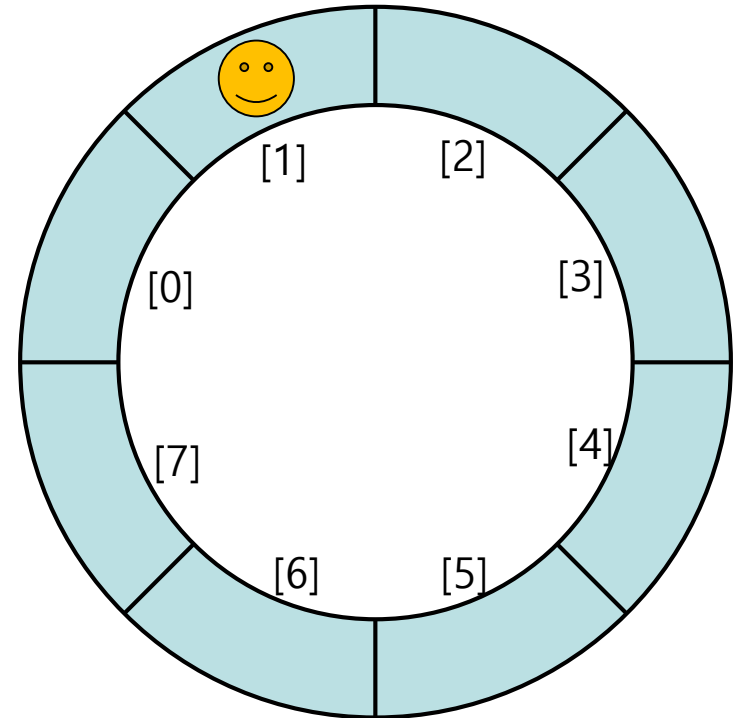
- 초기 시작은 (큐 empty 상태),
 - ◆ `this.front == 0`
 - ◆ `this.rear == 0`



□ Full 조건 [2]

■ 큐 Full 조건은?

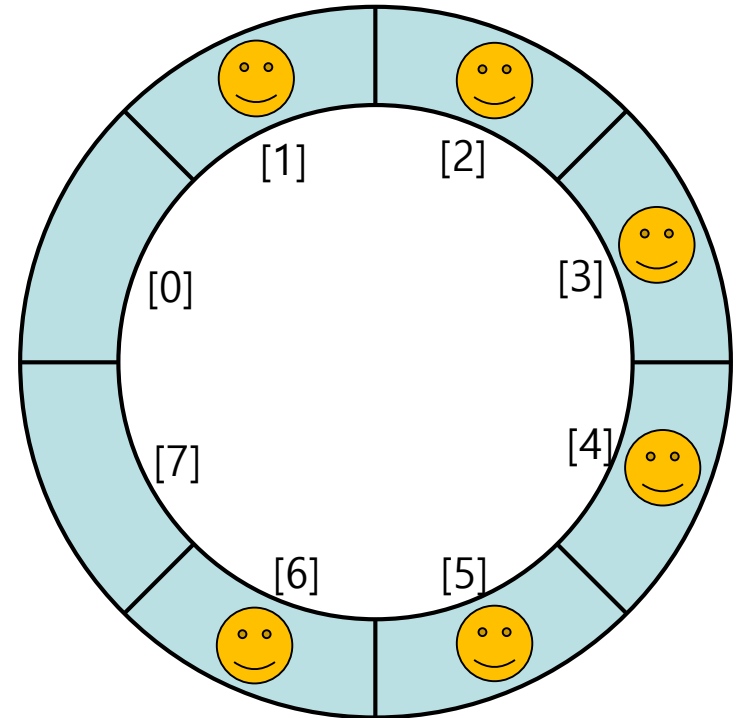
- 초기 시작은,
 - ◆ `this.front == 0`
 - ◆ `this.rear == 0`
- 1개가 차면,
 - ◆ `this.front == 0`
 - ◆ `this.rear == 1`



□ Full 조건 [2]

■ 큐 Full 조건은?

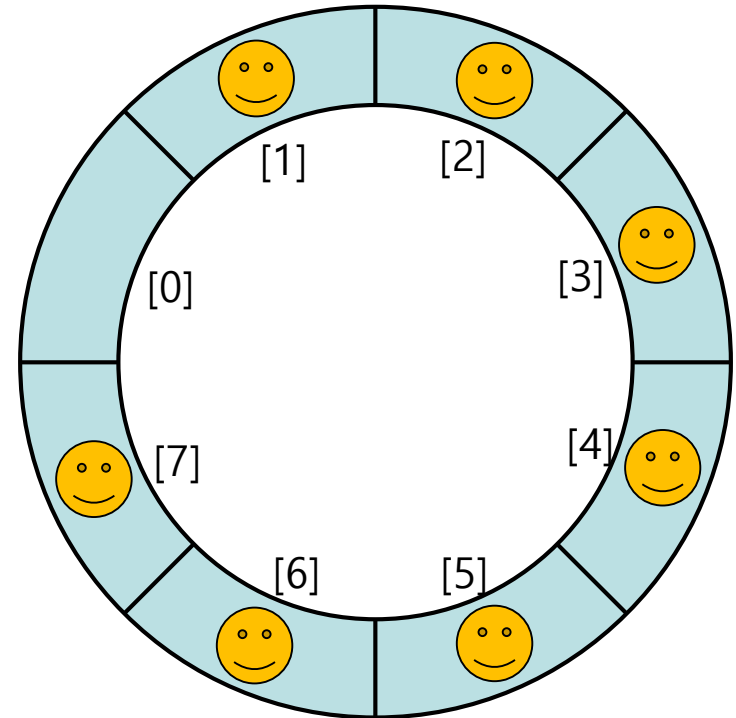
- 초기 시작은,
 - ◆ `this.front == 0`
 - ◆ `this.rear == 0`
- 6개가 차면,
 - ◆ `this.front == 0`
 - ◆ `this.rear == 6`



□ Full 조건 [2]

■ 큐 Full 조건은?

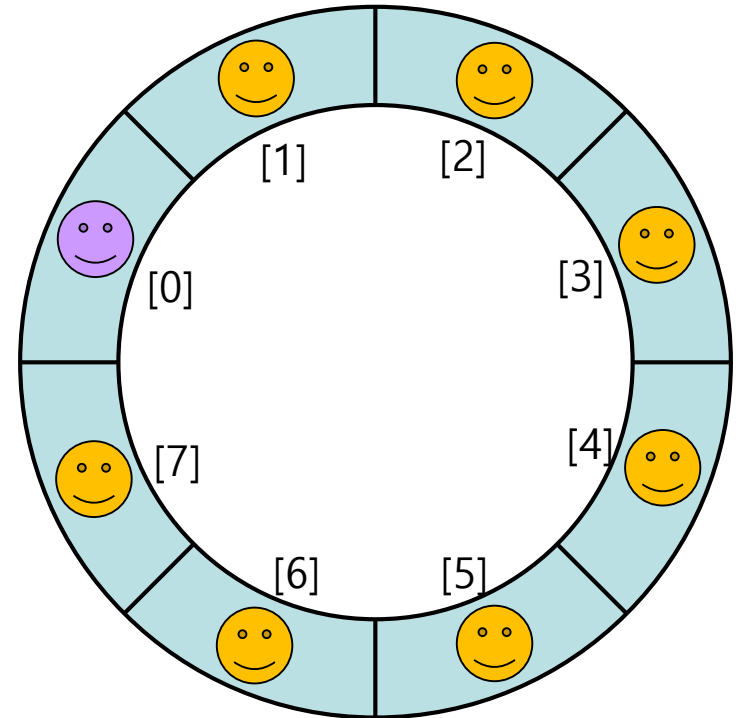
- 초기 시작은,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 0`
- 6개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 6`
- 7개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 7`



□ Full 조건 [3]

■ 큐 Full 조건은?

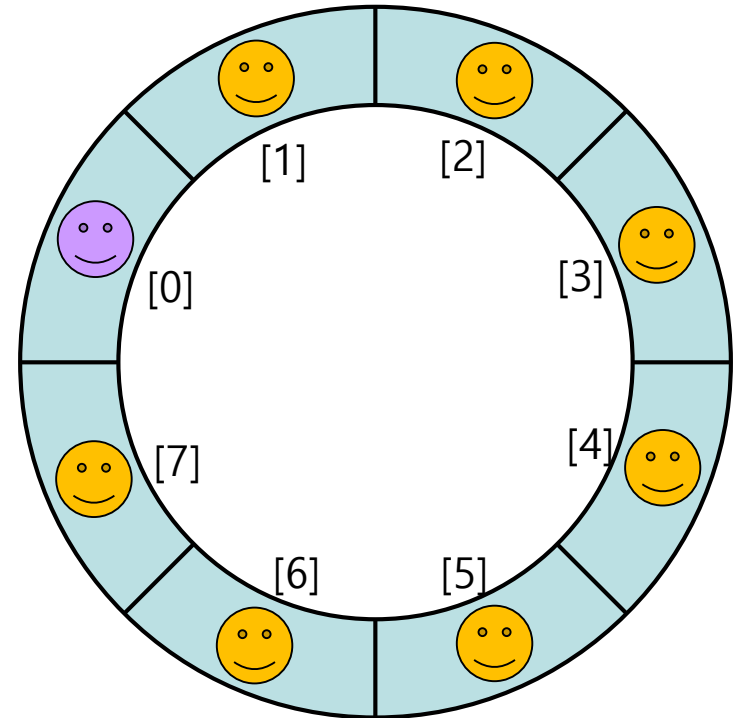
- 초기 시작은,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 0`
- 6개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 6`
- 7개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 7`
- 8개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 0`



□ Full 조건 [4]

■ 큐 Full 조건은?

- 초기 시작은,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 0`
- 6개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 6`
- 7개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 7`
- 8개가 차면,
 - ◆ `this._front == 0`
 - ◆ `this._rear == 0`



■ 모두 채우면 큐가 empty인지 full인지 구분 불가!

□ CircularArrayQueue<T>의 공개함수

■ CircularArrayQueue<T> 객체 사용법

- public CircularArrayQueue () ;
- public CircularArrayQueue (int initialCapacity) ;

- public boolean isEmpty () ;
- public boolean isFull () ;
- public int size () ;

- public T frontElement () ;

- public boolean enqueue (T anElement) ;
- public T dequeue () ;
- public void clear () ;

Class “CircularArrayQueue” 의 구현

❏ CircularArrayQueue: 비공개 인스턴스 변수

```
public class CircularArrayQueue<T>
{
    // 상수
    private static final int DEFAULT_CAPACITY = 50 ;

    // 비공개 인스턴스 변수
    private int      _capacity ;
    private int      _front ;
    private int      _rear ;
    private T[]      _elements ;
}
```

□ CircularArrayQueue의 생성자

```
public class CircularArrayQueue<T>
{
    // 생성자
    public CircularArrayQueue ()
    {
        this (CircularArrayQueue.DEFAULT_CAPACITY) ;
    }

    public CircularArrayQueue (int givenCapacity)
    {
        @SuppressWarnings("Unchecked") ;
        this._elements = (T[ ]) new Object[givenCapacity] ;
        this._capacity = givenCapacity ;
        this._front = 0 ;
        this._rear = 0 ;
    }
}
```

□ CircularArrayQueue : 상태 알아보기

```
public class CircularArrayQueue<T>
{
    .....

    // 상태 알아보기
    public boolean isEmpty () {
        return (this._front == this._rear) ;
    }

    public boolean isFull () {
        int nextRear = (this._rear + 1) % this._capacity ;
        return (nextRear == this._front) ;
    }

    public int size() {
        if ( this._front <= this._rear ) {
            return (this._rear - this._front)
        }
        else {
            return ( (this._rear + this._capacity) - this._front ) ;
        }
    }
}
```

❏ CircularArrayQueue : frontElement()

```
public T frontElement()  
{  
    T frontElement = null ;  
    if ( ! this.isEmpty() ) {  
        frontElement = this._elements[this._front] ;  
    }  
    return frontElement ;  
}
```

❑ CircularArrayQueue : enqueue()

// 원소 추가 함수

```
public boolean enqueue (T anElement)
{
    if ( this.isFull() ) {
        return false ;
    }
    else {
        this._rear = (this._rear+1) % this._capacity ;
        this._elements[this._rear] = anElement ;
        return true ;
    }
}
```

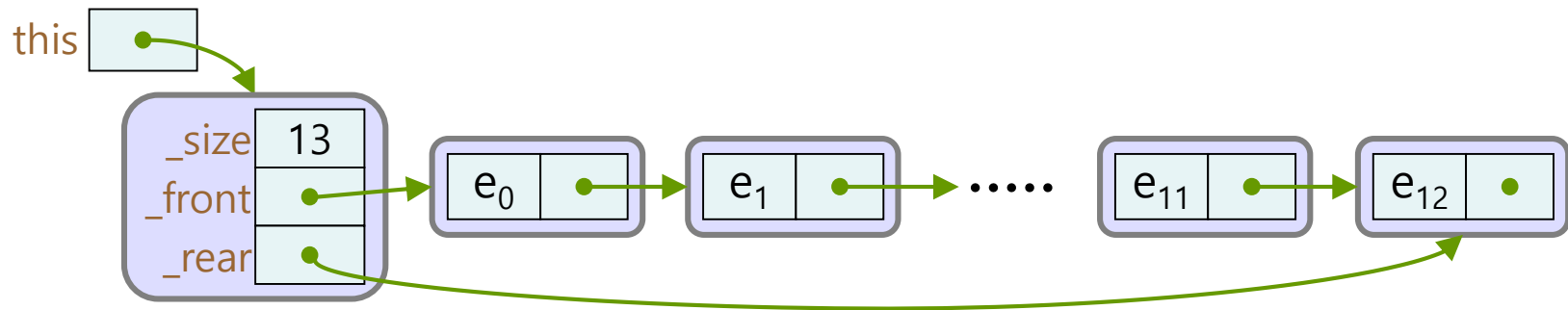
❑ CircularArrayQueue : dequeue()

```
public T dequeue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = _elements[this._front] ;
        this._elements[this._front] = null ;
        this._front = (this._front+1) % this._capacity ;
    }
    return frontElement ;
}
```

❑ CircularArrayQueue : clear()

```
public void clear ()  
{  
    for ( i = 0 ; i < this.size() ; i++ ) {  
        this._elements[(this._front) % this._capacity] = null ;  
    }  
    this._front = 0 ;  
    this._rear = 0 ;  
}
```

Class "LinkedListQueue"



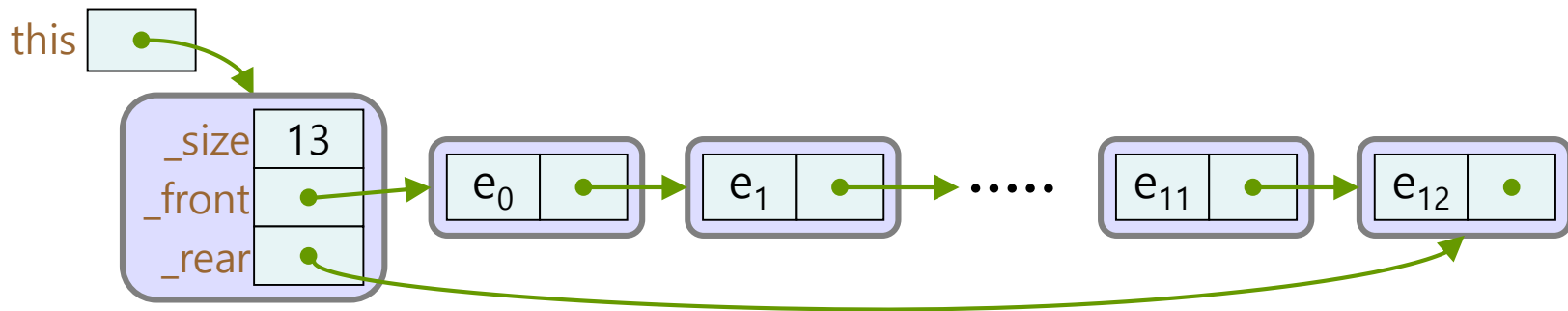
□ LinkedList<T> 의 공개함수

■ LinkedList<T> 객체 사용법

- public LinkedList() ;
- public boolean isEmpty() ;
- public boolean isFull () ;
- public int size();
- public T frontElement ();
- public boolean enqueue (T anElement) ;
- public T dequeue ();
- public void clear () ;

□ LinkedList<T> : 인스턴스 변수

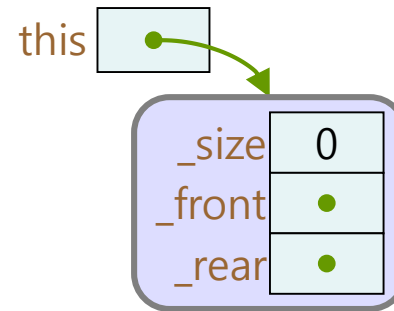
```
public class LinkedList<T>
{
    // 비공개 인스턴스 변수
    private int    _size ;
    private Node<T> _front ;
    private Node<T> _rear ;
}
```



□ Class “LinkedListQueue”의 구현: 생성자

```
public class LinkedListQueue<T>
{
    // 비공개 멤버 변수
    .....

    // 생성자
    public LinkedListQueue ( )
    {
        this._size = 0 ;
        this._front = null ;
        this._rear = null ;
    }
}
```



□ LinkedList : 상태 알아보기

```
public class LinkedList<T>
{
    // 비공개 멤버 변수
    .....

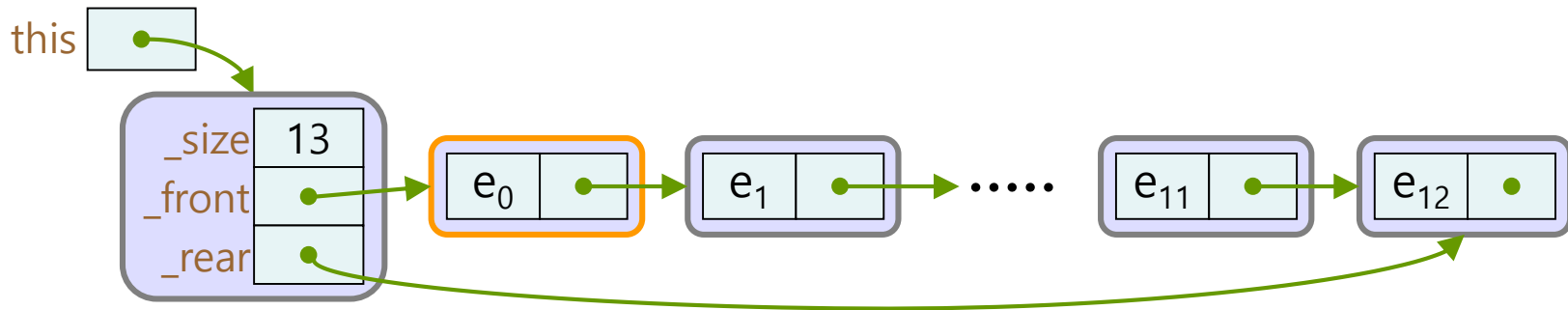
    // Queue가 비어있는지 확인
    public boolean isEmpty ()
    {
        return (this._front == null) && (this._rear == null);
        // 또는 간단히: return (this._front == null) ;
    }

    public boolean isFull ()
    {
        return false ;
    }

    public int size ()
    {
        return this._size ;
    }
}
```

❏ LinkedList : frontElement ()

```
public T frontElement()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._front.element() ;
    }
    return frontElement ;
}
```

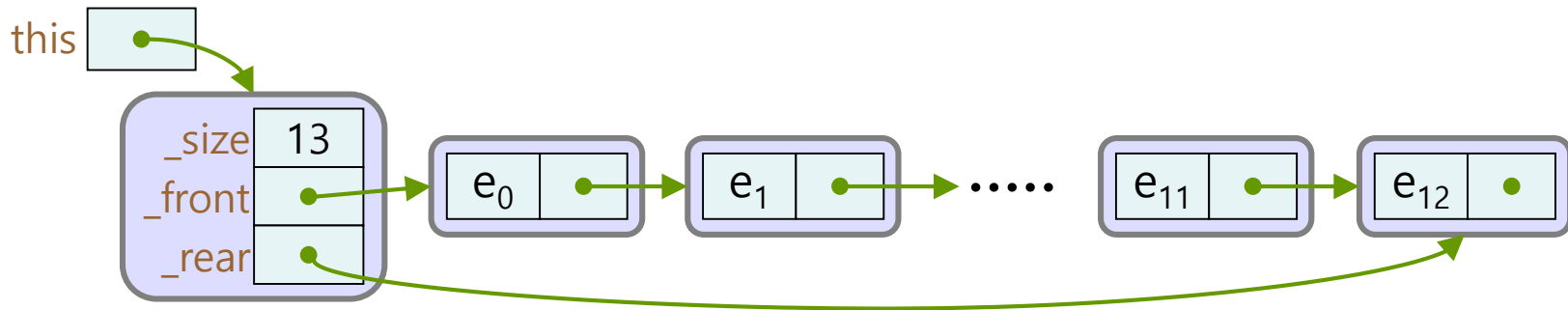


❏ LinkedList: enqueue()

```

public void enqueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        this._front = newRearNode ;
    }
    else {
        this._rear.setNext (newRearNode) ;
    }
    this._rear = newRearNode ;
    this._size++ ;
}

```

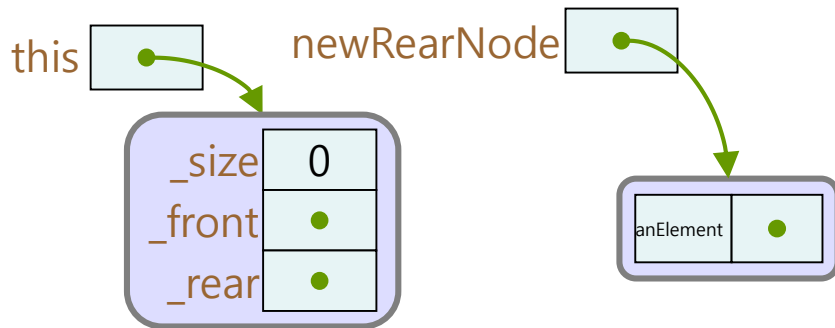


❏ enqueue() [1]

```

public void enqueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        this._front = newRearNode ;
    }
    else {
        this._rear.setNext (newRearNode) ;
    }
    this._rear = newRearNode ;
    this._size++ ;
}

```

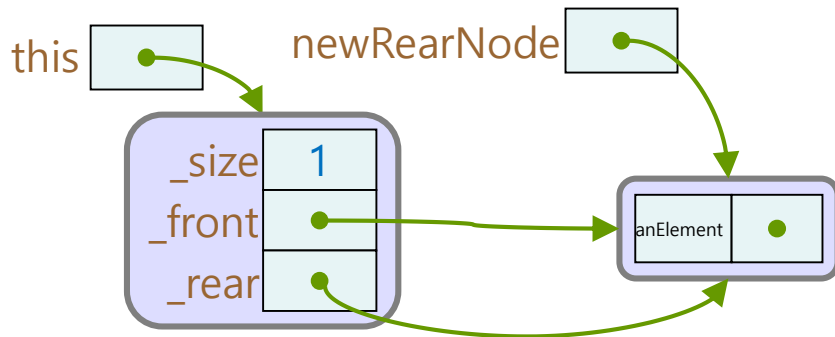


enQueue() [2]

```

public void enQueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        this._front = newRearNode ;
    }
    else {
        this._rear.setNext (newRearNode) ;
    }
    this._rear = newRearNode ;
    this._size++ ;
}

```

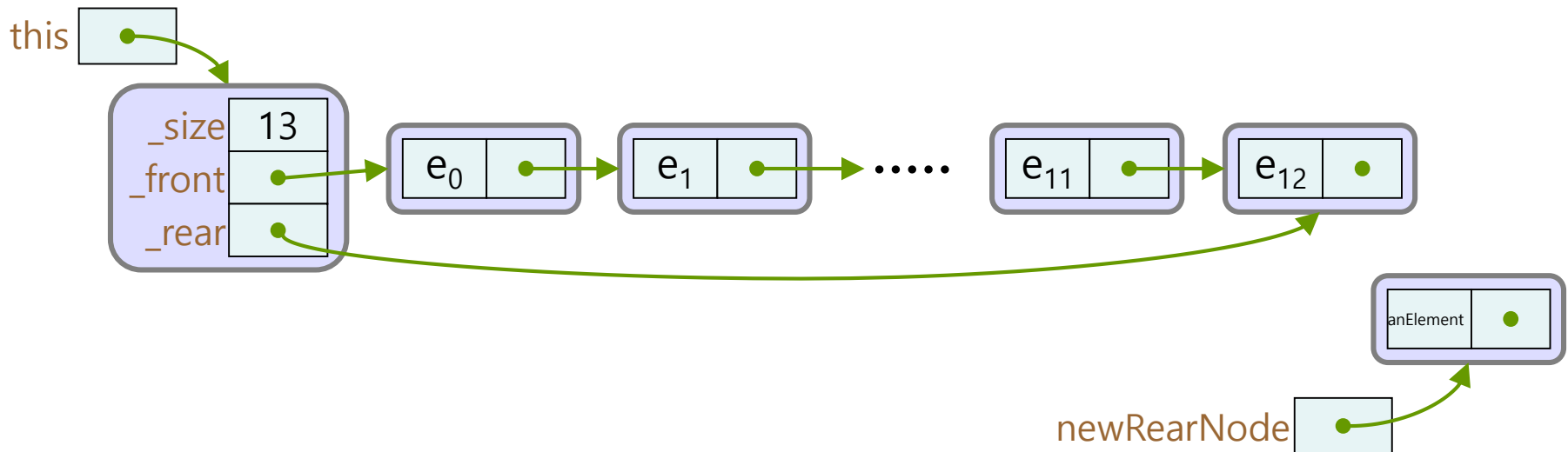


enQueue() [3]

```

public void enQueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        this._front = newRearNode ;
    }
    else {
        this._rear.setNext (newRearNode) ;
    }
    this._rear = newRearNode ;
    this._size++ ;
}

```

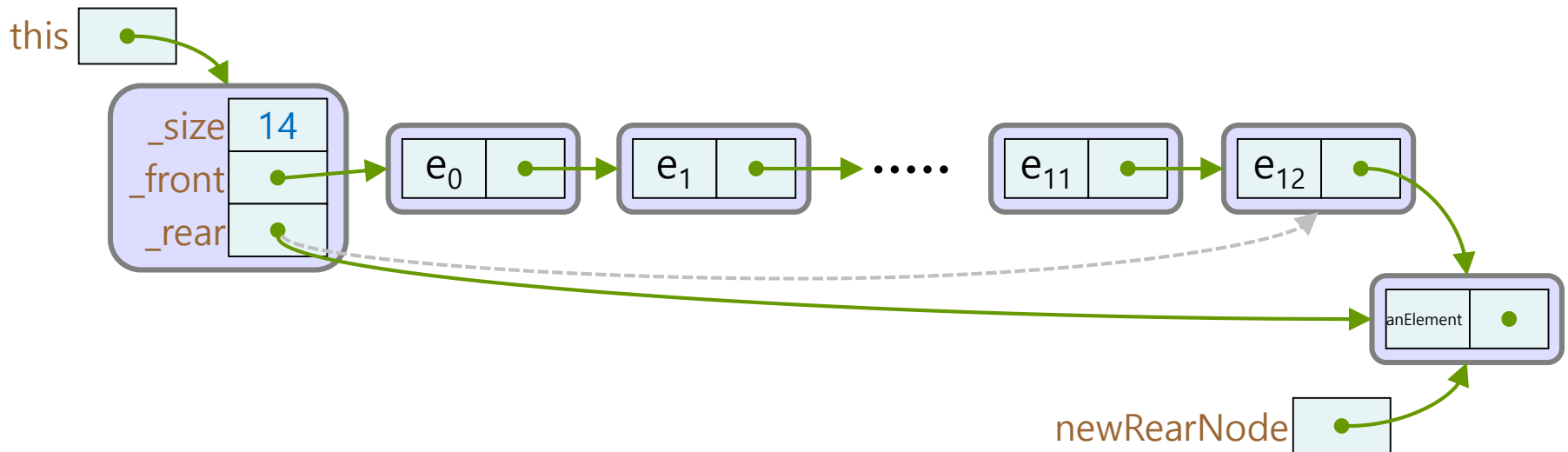


enQueue() [4]

```

public void enQueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        this._front = newRearNode ;
    }
    else {
        this._rear.setNext (newRearNode) ;
    }
    this._rear = newRearNode ;
    this._size++ ;
}

```

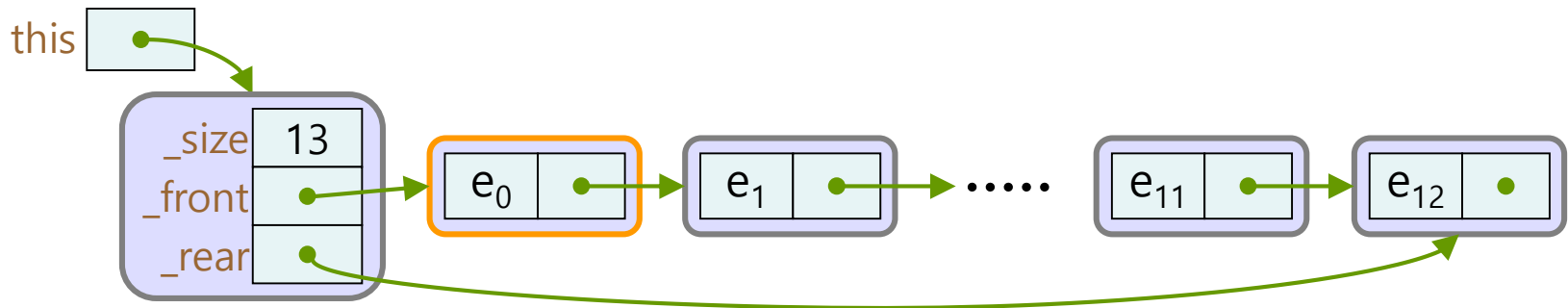


❏ LinkedList : dequeue()

```

public T dequeue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._front.element() ;
        this._front = this._front.next() ;
        if ( this._front == null ) {
            this._rear = null;
        }
        this._size--;
    }
    return frontElement ;
}

```

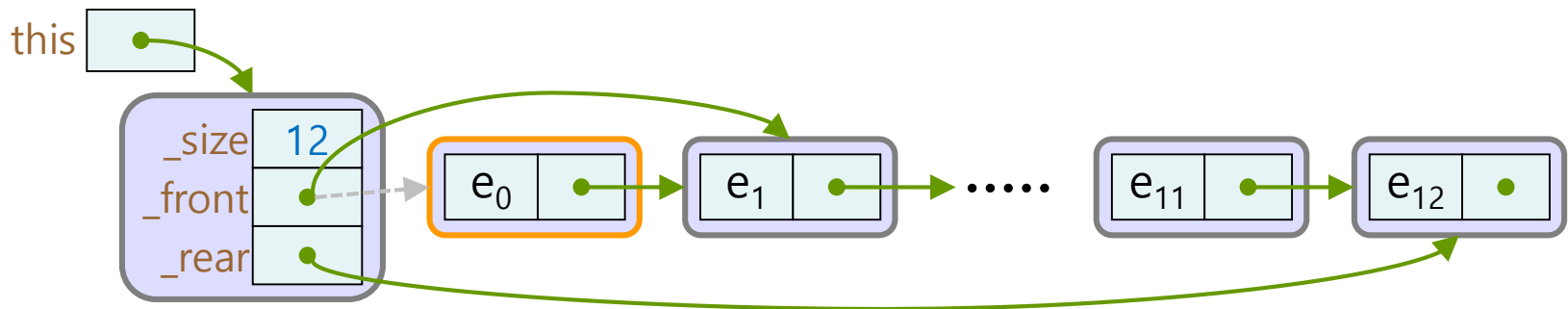


❏ deQueue() [1]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._front.element() ;
        this._front = this._front.next() ;
        if ( this._front == null ) {
            this._rear = null;
        }
        this._size--;
    }
    return frontElement ;
}

```

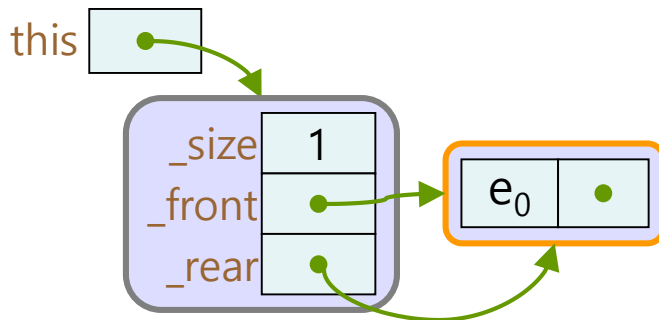


❏ deQueue() [2]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._front.element() ;
        this._front = this._front.next() ;
        if ( this._front == null ) {
            this._rear = null;
        }
        this._size--;
    }
    return frontElement ;
}

```



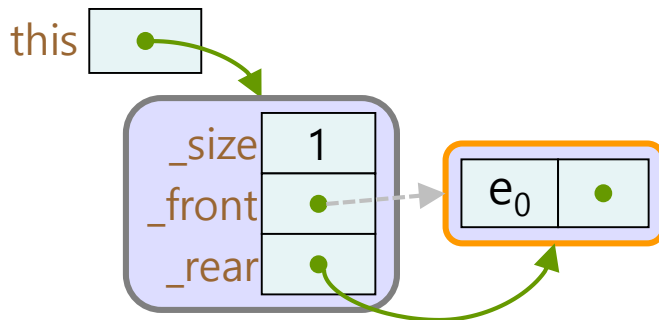
❏ deQueue() [3]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._front.element() ;
        this._front = this._front.next() ;
        if ( this._front == null ) {
            this._rear = null;
        }
        this._size--;
    }
    return frontElement ;
}

```

_front의 값은 null이 된다

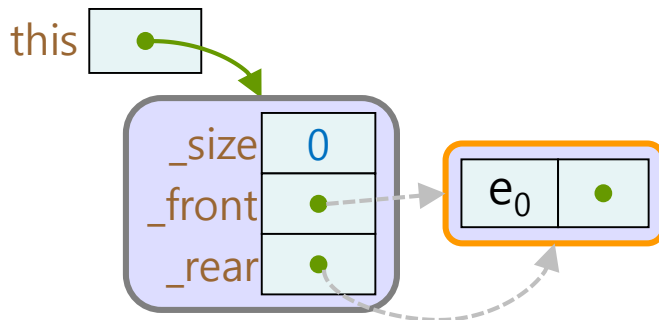


❏ deQueue() [4]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._front.element() ;
        this._front = this._front.next() ;
        if ( this._front == null ) {
            this._rear = null;
        }
        this._size--;
    }
    return frontElement ;
}

```

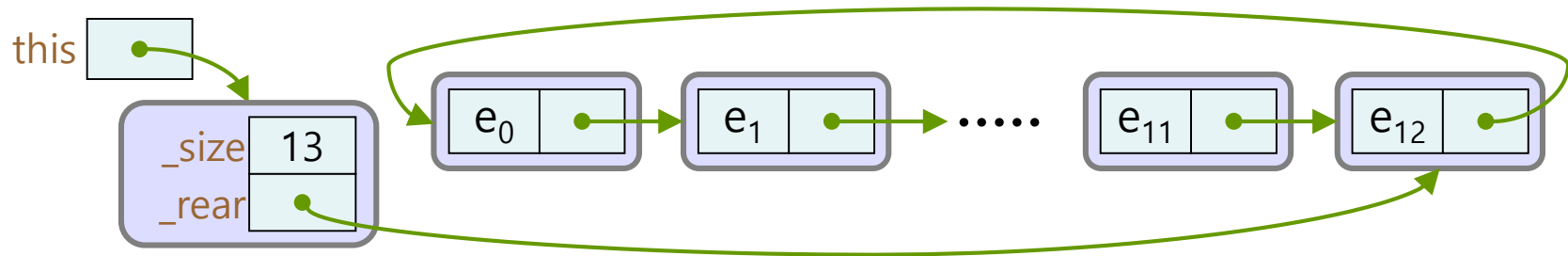


LinkedList : clear()

```
public void clear()  
{  
    this._front = null ;  
    this._rear = null ;  
    _size = 0 ;  
}
```

Class

"CircularlyLinkedQueue"



□ CircularlyLinkedListQueue의 공개함수

■ CircularlyLinkedListQueue<T> 객체 사용법

- public CircularlyLinkedListQueue() ;
- public boolean isEmpty () ;
- public boolean isFull () ;
- public int size () ;
- public T frontElement ();
- public boolean enqueue (T anElement) ;
- public T dequeue ();

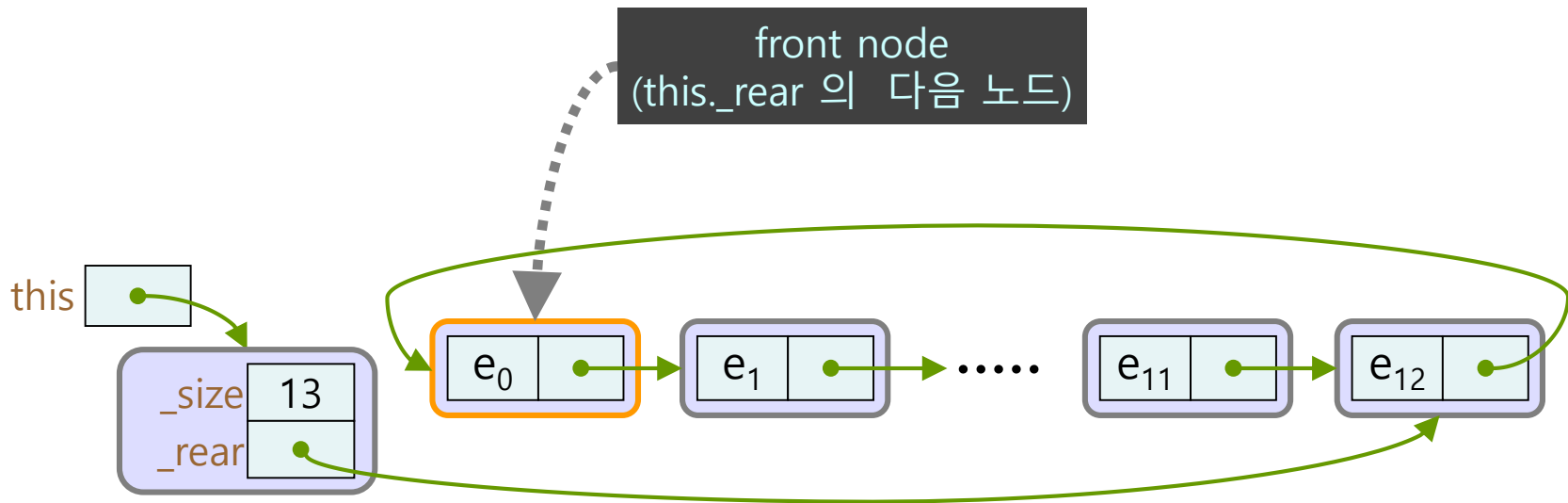
❑ CircularlyLinkedListQueue: 비공개 인스턴스 변수

```
public class CircularlyLinkedListQueue<T>
{
```

```
    // 비공개 인스턴스 변수
```

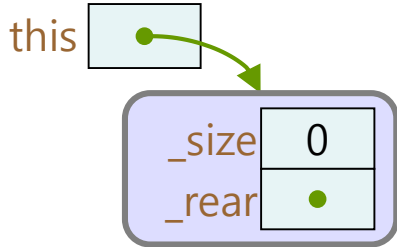
```
    private int      _size ;
```

```
    private Node<T> _rear ;
```



□ CircularlyLinkedListQueue: 생성자

```
public class CircularlyLinkedListQueue<T>
{
    // 생성자
    public circularlyLinkedListQueue( )
    {
        this._size = 0 ;
        this._rear = null ;
    }
}
```



□ CircularlyLinkedListQueue: 상태 알아보기

```
public class CircularlyLinkedListQueue<T>  
{
```

```
.....
```

```
// 상태 알아보기
```

```
public boolean isEmpty ()  
{  
    return ( this._rear ==null ) ;  
}
```

```
public boolean isFull ()  
{  
    return false ;  
}
```

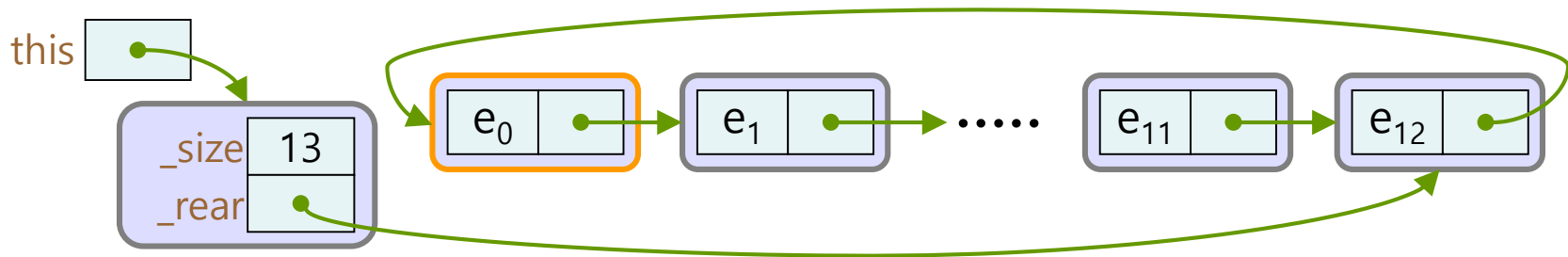
```
public int size ()  
{  
    return this._size ;  
}
```

□ CircularlyLinkedQueue: frontElement()

```

public T frontElement ()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._rear.next().element() ;
    }
    return frontElement ;
}

```

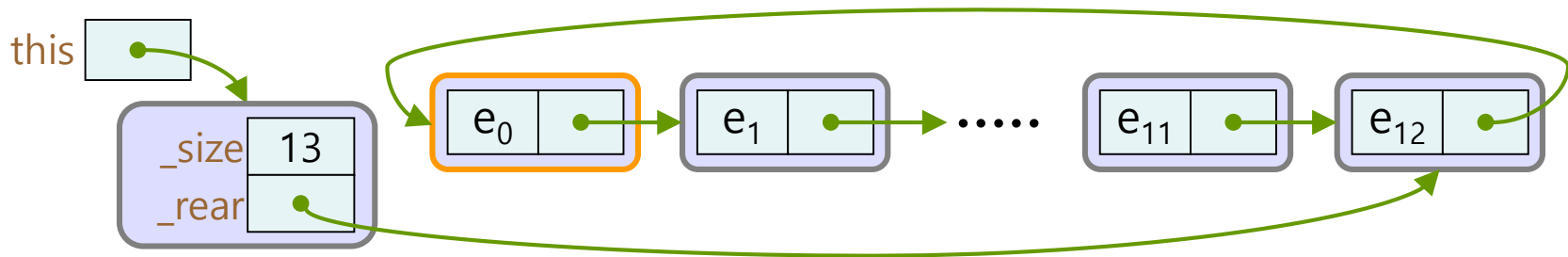


□ CircularlyLinkedListQueue : enqueue()

```

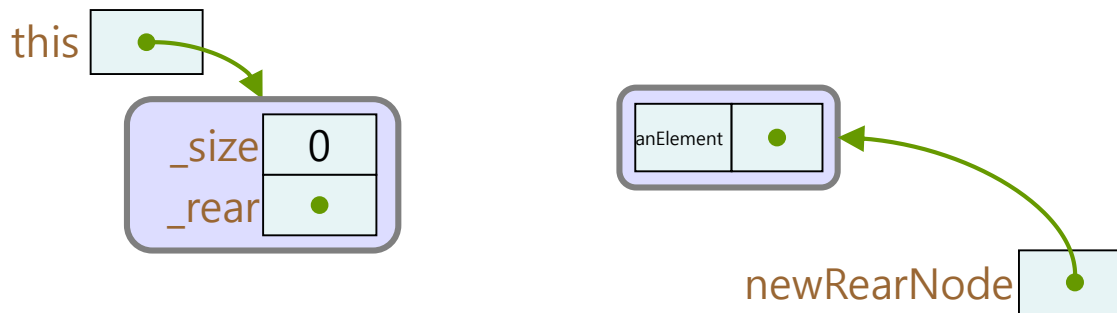
public void enqueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        newRearNode.setNext (newRearNode) ;
    }
    else {
        newRearNode.setNext (this._rear.next()) ;
        this._rear.setNext (newRearNode) ;
    }
    this._rear= newRearNode ;
    this._size++ ;
}

```



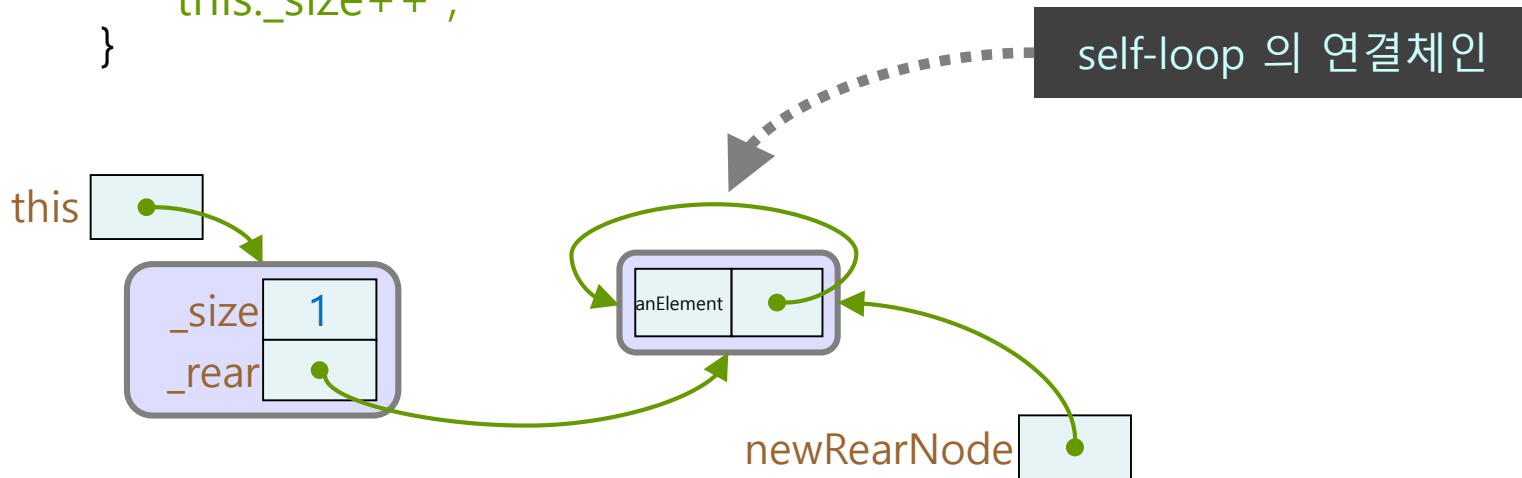
❏ enqueue() [0]

```
public void enqueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        newRearNode.setNext (newRearNode) ;
    }
    else {
        newRearNode.setNext (this._rear.next()) ;
        this._rear.setNext (newRearNode) ;
    }
    this._rear= newRearNode ;
    this._size++ ;
}
```



❏ enqueue() [1]

```
public void enqueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        newRearNode.setNext (newRearNode) ;
    }
    else {
        newRearNode.setNext (this._rear.next()) ;
        this._rear.setNext (newRearNode) ;
    }
    this._rear= newRearNode ;
    this._size++ ;
}
```

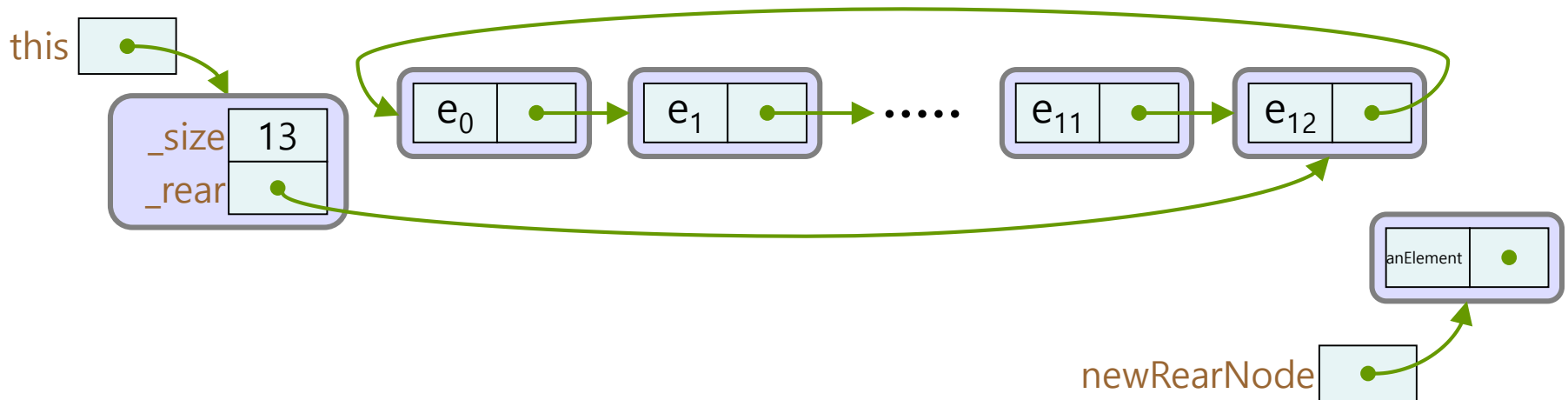


enQueue() [2]

```

public void enQueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        newRearNode.setNext (newRearNode) ;
    }
    else {
        newRearNode.setNext (this._rear.next()) ;
        this._rear.setNext (newRearNode) ;
    }
    this._rear= newRearNode ;
    this._size++ ;
}

```

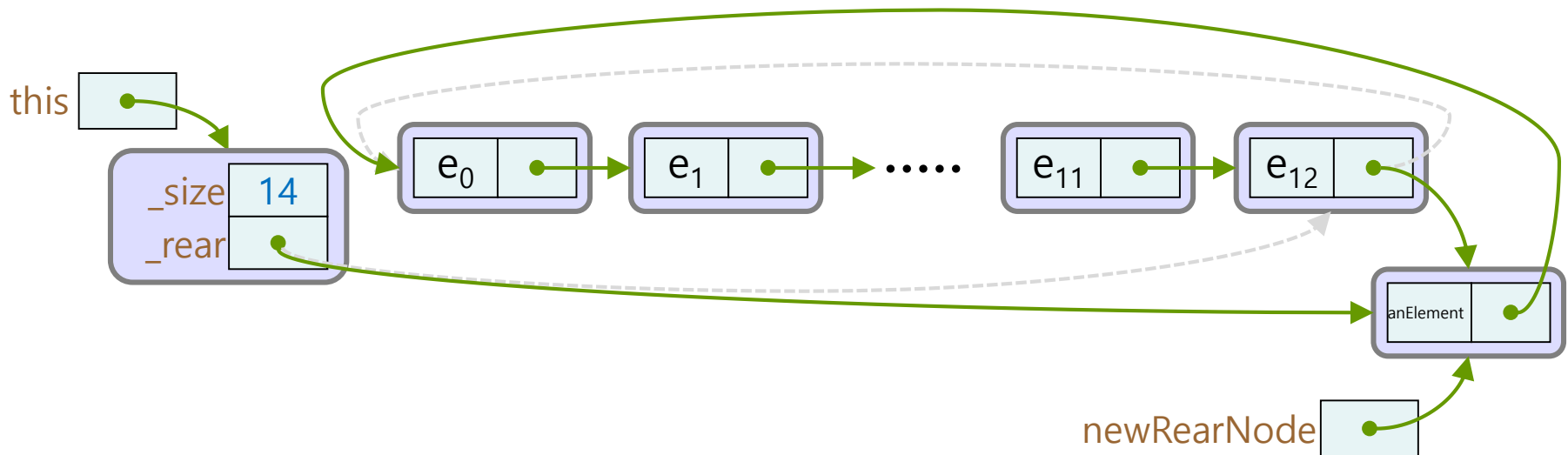


enQueue() [3]

```

public void enQueue (T anElement)
{
    Node<T> newRearNode = new Node (anElement, null) ;
    if ( this.isEmpty() ) {
        newRearNode.setNext (newNode) ;
    }
    else {
        newRearNode.setNext (this._rear.next()) ;
        this._rear.setNext (newRearNode) ;
    }
    this._rear= newRearNode ;
    this._size++ ;
}

```

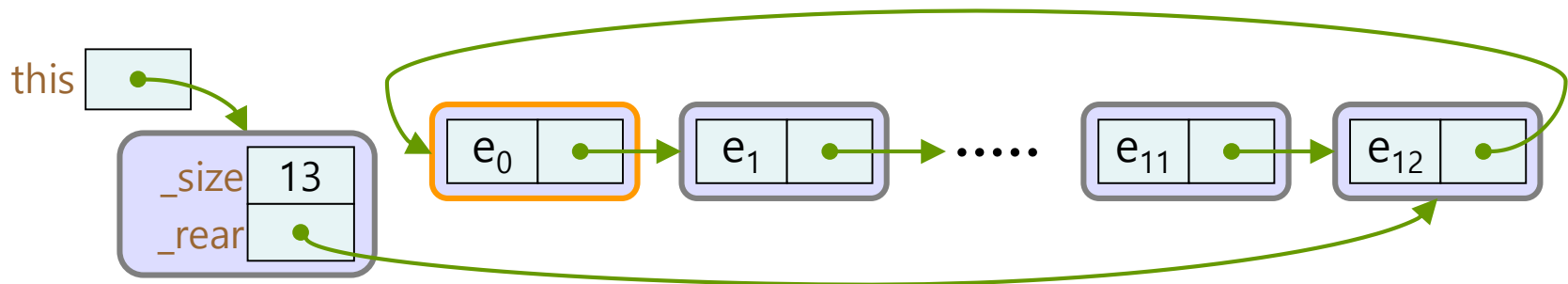


□ CircularlyLinkedQueue: dequeue()

```

public T dequeue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._rear.next().element() ;
        if ( this._rear == this._rear.next() ) { // 노드가 한 개: self-loop의 경우
            this._rear = null ;
        }
        else { // 노드가 2 개 이상
            this._rear.setNext() = this._rear.next().next() ;
        }
        this._size-- ;
    }
    return frontElement ;
}

```

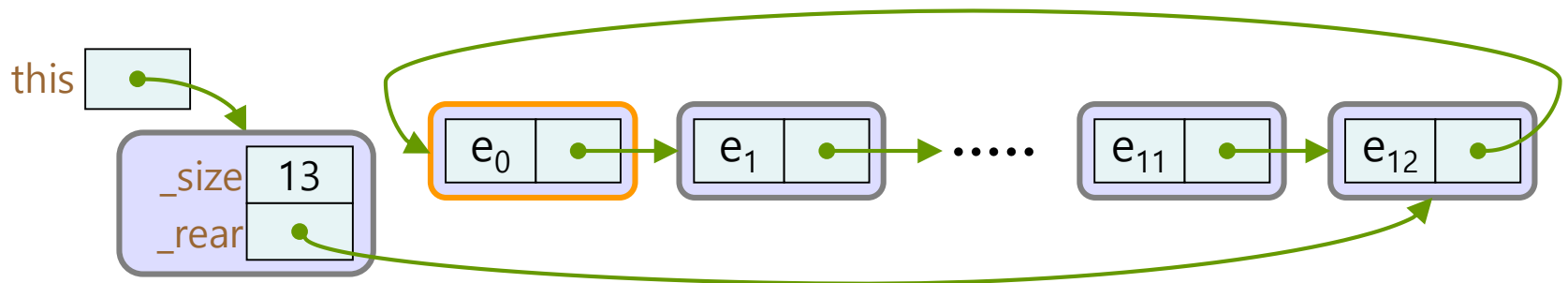


❏ deQueue() [1]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._rear.next().element() ;
        if ( this._rear == this._rear.next() ) { // 노드가 한 개: self-loop의 경우
            this._rear = null ;
        }
        else { // 노드가 2 개 이상
            this._rear.setNext() = this._rear.next().next() ;
        }
        this._size-- ;
    }
    return frontElement ;
}

```

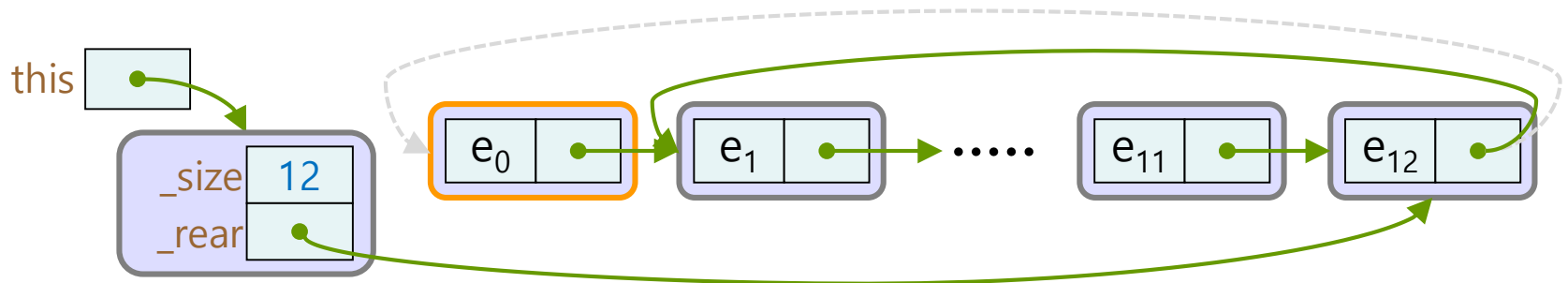


❏ deQueue() [2]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._rear.next().element() ;
        if ( this._rear == this._rear.next() ) { // 노드가 한 개: self-loop의 경우
            this._rear = null ;
        }
        else { // 노드가 2 개 이상
            this._rear.setNext() = this._rear.next().next() ;
        }
        this._size-- ;
    }
    return frontElement ;
}

```

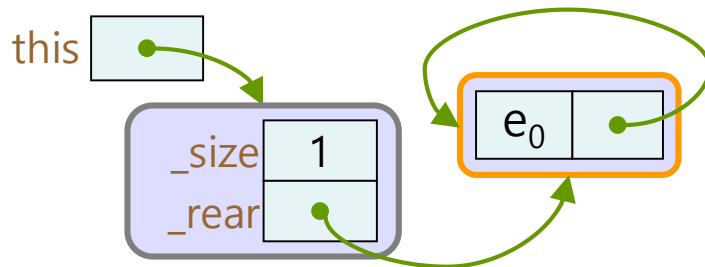


❏ deQueue() [3]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._rear.next().element() ;
        if ( this._rear == this._rear.next() ) { // 노드가 한 개: self-loop의 경우
            this._rear = null ;
        }
        else { // 노드가 2 개 이상
            this._rear.setNext() = this._rear.next().next() ;
        }
        this._size-- ;
    }
    return frontElement ;
}

```

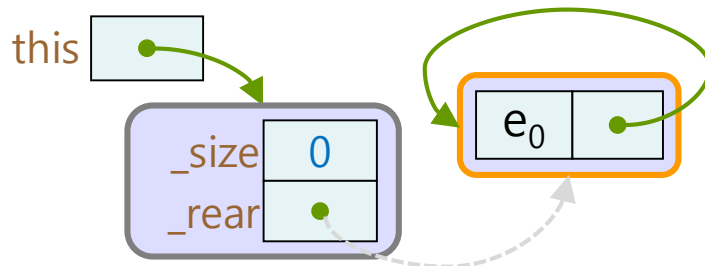


❏ deQueue() [4]

```

public T deQueue()
{
    T frontElement = null ;
    if ( ! this.isEmpty() ) {
        frontElement = this._rear.next().element() ;
        if ( this._rear == this._rear.next() ) { // 노드가 한 개: self-loop의 경우
            this._rear = null ;
        }
        else { // 노드가 2 개 이상
            this._rear.setNext() = this._rear.next().next() ;
        }
        this._size-- ;
    }
    return frontElement ;
}

```



LinkedList : clear()

```
public void clear()  
{  
    this._front = null ;  
    this._rear = null ;  
    _size = 0 ;  
}
```

실습:

Simulation Of Waiting Line

□ 실습: Simulation Of Waiting Line

- Queue 를 이용하여 대기 열 (Waiting Line) 프로그램을 작성한다.
- 각각 손님마다 일이 처리 되는데 걸리는 시간은 다르다.
- 손님이 왔을 때 대기 시간과 현재 대기 인원을 출력한다.

“큐” [끝]



